

Dynamic Pruning of Learned Sparse Representations using Cluster-Partitioned Inverted Indexes

Roger Canal Estrada

Universitat Politècnica de Catalunya (UPC)
Facultat d'Informàtica de Barcelona (FIB)
`roger.canal@estudiantat.upc.edu`

Abstract. Learned sparse representations offer state-of-the-art retrieval effectiveness while retaining the interpretability of lexical models. However, because these embeddings lack the Zipfian distribution of traditional term frequencies, standard inverted index pruning algorithms suffer severe performance degradation. In this paper, I propose an approximate retrieval system that partitions the inverted index using global document clustering. By grouping semantically similar documents, it computes block-level summary vectors that enable aggressive dynamic pruning during query evaluation. I detail the system’s core components: an indexing pipeline and a highly optimized search engine. Preliminary experiments on the 2.68M document Natural Questions (NQ) dataset demonstrate the system’s capacity for extreme low-latency retrieval (1.5ms), and I highlight the structural tuning required to balance this latency with high recall.

Keywords: Sparse Retrieval · Approximate Nearest Neighbor Search · Dynamic Pruning · Learned Sparse Representations

1 Introduction

The emergence of Learned Sparse Representations (LSR), such as SPLADE [4], revitalizes inverted index-based retrieval. By mapping contextualized embeddings back into the vocabulary space, LSR mitigates the vocabulary mismatch problem while allowing retrieval to be formulated as a Maximum Inner Product Search (MIPS). Despite this, executing MIPS efficiently over LSRs is challenging. Traditional dynamic pruning algorithms, such as WAND [1], assume a Zipfian distribution of term weights. LSRs fundamentally violate this assumption by heavily activating a broader, flatter set of dimensions, leading to a phenomenon where standard score-at-a-time evaluation strategies fail to prune effectively.

To address this latency bottleneck, approximate retrieval strategies are required. Recent works, such as SEISMIC, demonstrate that grouping documents into geometrically cohesive blocks and computing summary upper bounds drastically reduces the search space [2]. Extensions of this paradigm successfully pair

such partitioned indexes with κ -NN graphs [3] and prove scalable to massive collections.

In this work, I present an implementation of an approximate sparse search pipeline built entirely in C++, drawing inspiration from these recent advancements. My approach relies on partitioning the high-dimensional space via K-means clustering, building a truncated partitioned inverted index, and applying a fast pruning algorithm. I outline the architectural details of my implementation and evaluate its performance on a massive-scale dataset.

2 System Architecture

The system consists of an indexing pipeline and a search engine. The indexing pipeline is subdivided into a global document clustering phase and a partitioned index construction phase.

2.1 Document Clustering Phase

The first step of the pipeline applies a sparse K-means-style clustering algorithm to the entire document collection. Using random document initialization for centroids, the algorithm assigns each sparse document vector to the cluster centroid that yields the highest inner product. The system then updates the centroids by summing and normalizing the sparse document vectors assigned to them.

This phase introduces the K hyperparameter, which dictates the total number of clusters. The purpose of this phase is not to route queries directly to a single cluster, but to create geometrical cohesiveness within the inverted lists. By grouping semantically related documents, the maximum weight of a vocabulary concept within a specific cluster is tightly bounded. With this clustering, an inverted list contains a list of blocks with related documents, thereby opening the opportunity for dynamic pruning.

2.2 Partitioned Index

Once the system clusters the documents, it computes a “summary vector” for each cluster. For every concept (dimension), the summary vector stores a representative weight (the maximum weight seen among all documents in that cluster). Algorithm 1 outlines the construction of this Partitioned Index.

Instead of storing a list of documents for each concept, the index is represented as an array of concepts, where each concept contains a list of **InvertedBlock** structures. To enforce a baseline level of approximation, the index undergoes truncation at build-time using two hyperparameters: nb (maximum blocks retained per dimension) and nd (maximum documents retained per block).

Algorithm 1 Partitioned Index Construction

Require: Clustered documents, nb (max blocks), nd (max docs)

Ensure: Partitioned Inverted Index I

```
1: for each concept  $c \in Vocabulary$  do
2:    $ClusterWeights \leftarrow$  Calculate summary weight of  $c$  for each cluster
3:    $TopClusters \leftarrow$  Select top  $nb$  clusters from  $ClusterWeights$ 
4:   for each cluster  $k \in TopClusters$  do
5:      $TopDocs \leftarrow$  Select top  $nd$  documents in  $k$  having concept  $c$ 
6:      $Block \leftarrow$  Create InvertedBlock( $k, TopDocs$ )
7:     Append  $Block$  to  $I[c]$ 
8:   end for
9: end for
10: return  $I$ 
```

2.3 Search Engine

To execute queries efficiently, the Search Engine relies on Coordinate-at-a-Time (CaaT)¹ traversal, query-side reorganization, and aggressive dynamic pruning.

When the engine receives a query, it dynamically reorganizes the search path to guarantee that the best candidate documents enter the Top- K results heap immediately. First, the algorithm sorts the non-zero query coordinates descending by weight, ensuring the most discriminative terms drive the initial search. Second, it calculates a theoretical upper bound (**cluster_ub**) for each block via the pre-computed summary vectors.

Algorithm 2 details this logic. During traversal, if the **cluster_ub** falls below the heap minimum relaxed by a **heap_factor** hyperparameter, the engine instantly skips the entire block without executing any exact dot products. Furthermore, the engine respects a hard early-termination limit, md (maximum documents examined), to bound worst-case latency.

3 Experimental Evaluation

I evaluate the system on the 2.68 million document Natural Questions (NQ) dataset. The embeddings derive from SPLADE with 30,522 dimensions. To measure retrieval quality, I calculate Recall@30 by comparing the approximate results of the search engine against the exact k -nearest neighbors (k -NN) gold standard provided within the dataset.

3.1 Hyperparameter Configuration

To properly frame the evaluation, an execution is defined by the hyperparameters introduced in the previous section:

¹ In learned sparse retrieval, Term-at-a-Time (TaaT) scoring is often called Coordinate-at-a-Time (CaaT) because terms act as coordinates in a high-dimensional vector space, contrasting with Document-at-a-Time (DaaT) traversal.

Algorithm 2 Search Engine

Require: Query q , Index I , Summaries, **heap_factor**, md , $Results_K$ **Ensure:** Top- $Results_K$ documents

```
1:  $SortedTerms \leftarrow$  Sort non-zero terms in  $q$  by weight descending
2:  $Heap \leftarrow$  Initialize Min-Heap of size  $Results\_K$ 
3:  $ClusterUB \leftarrow$  Compute upper bounds for all clusters using  $q$  and Summaries
4:  $DocsExamined \leftarrow 0$ 
5: for each term  $t \in SortedTerms$  do
6:   for each InvertedBlock  $B \in I[t]$  do
7:     if  $ClusterUB[B.id] < Heap.min() \times \text{heap\_factor}$  then
8:       continue ▷ Aggressively skip entire block
9:     end if
10:    for each document  $d \in B.docs$  do
11:      if  $DocsExamined \geq md$  then
12:        return  $Heap$ 
13:      end if
14:       $score \leftarrow \text{ExactDotProduct}(q, d)$ 
15:      Update  $Heap$  with  $(score, d)$ 
16:       $DocsExamined \leftarrow DocsExamined + 1$ 
17:    end for
18:  end for
19: end for
20: return  $Heap$ 
```

- K : The number of global clusters.
- nb & nd : The index breadth (max blocks) and depth (max docs per block).
- md : The hard cap on total documents examined during search.
- **heap_factor**: The relaxation multiplier governing dynamic block skipping.

3.2 Current Results

Table 1 showcases a preliminary subset of experiments focused on optimizing for extreme low latency. In these specific configurations, the index restricts its breadth ($nb \in \{150, 175, 200\}$) and applies a strict pruning threshold (**heap_factor** = 1.0).

Table 1. Preliminary performance metrics on the NQ dataset focusing on ultra-low latency heuristics.

K (Clusters)	nb	nd	md	Docs Examined	Blocks Skipped	Latency (ms)	Recall@30
2500	150	80	30000	1,011	4,030	1.52	0.370
2500	150	100	30000	1,158	4,031	1.58	0.375
2500	175	100	30000	1,164	4,705	1.62	0.379
2500	200	80	30000	1,020	5,379	1.58	0.378
2500	200	90	30000	1,095	5,380	1.57	0.380

This “optimized” configuration successfully drives average query latency down to an exceptional ~ 1.5 ms. However, this speed causes “search starvation.” Because the `heap_factor` allows the engine to skip over 4,000 blocks per query, the system examines barely 1,000 documents out of the 2.68M available, restricting Recall@30 to roughly 0.37.

3.3 Incoming Results

To achieve the target Recall@30 of ≥ 0.90 , my incoming experiments focus on decoupling index quality from the search pruning heuristic. Specifically, I am configuring executions to lower the `heap_factor` (e.g., to 0.5 or 0.6). This softens the skip barrier, forcing the engine to evaluate a broader subset of highly-ranked blocks (examining $\sim 3,000$ to 5,000 documents) to recapture the gold-standard items missed by overly aggressive pruning. Concurrently, I am scaling the index breadth ($nb \geq 300$) to ensure a wider coverage of the 2.68M document dataset during the offline build phase, aiming to stabilize the optimal trade-off point at under 10ms latency.

4 Conclusion and Future Work

I detail an implemented approximate retrieval system for learned sparse vectors. By clustering documents and partitioning the inverted index, the search engine prioritizes query coordinates to raise pruning thresholds early, allowing it to aggressively skip vast portions of the dataset. My preliminary evaluations demonstrate the system’s capacity for sub-2ms latencies, though at a temporary cost to recall due to search starvation. Future work focuses on tuning the `heap_factor` and `nb` parameters to find the optimal sweet spot, alongside memory-aligning the forward index to ensure contiguous cache hits during document evaluation.

References

1. Broder, A. Z., Carmel, D., Herscovici, M., Soffer, A., & Zien, J. (2003). Efficient query evaluation using a two-level retrieval process. In *Proceedings of the 12th International Conference on Information and Knowledge Management (CIKM)*.
2. Bruch, S., Nardini, F. M., Rulli, C., & Venturini, R. (2024). Efficient Inverted Indexes for Approximate Retrieval over Learned Sparse Representations. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*.
3. Bruch, S., Nardini, F. M., Rulli, C., & Venturini, R. (2024). Pairing Clustered Inverted Indexes with κ -NN Graphs for Fast Approximate Retrieval over Learned Sparse Representations. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM)*.
4. Formal, T., Piwowarski, B., & Clinchant, S. (2021). SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*.